Book Catalog

You will build a REPL ("read-eval-print-loop", i.e. a command loop, like a tiny shell) to manage a catalog of books. You implement linear and binary search, and sorting algorithms of your choice.

Deliverables

What you should turn in:

- Your completed and added source files (everything marked `TODO`).
- Short written answers to the design questions at the end of this document, a paragraph each. One pdf or plain text file, submitted alongside your code.

How the project is organized

The code is split into four **packages**, one per folder: `catalog`, `order`, `sorting`, `searching`. The folder name *is* the package name: a file in `sorting/` starts with `package sorting;`, and code elsewhere reaches it with `import sorting.SortingAlgorithm;`. `Main.java` sits at the top, outside any package.

File	Status
`Main.java`	given, complete: the main REPL loop
`catalog/Book.java`	given, complete
`catalog/Catalog.java`	given, **except** `loadFromFile`
`catalog/Range.java`	`TODO`
`order/BookOrder.java`, `ById.java`, `ByTitle.java`, `ByAuthor.java`, `ByYear.java`	given, complete
`sorting/SortingAlgorithm.java`	given, complete, abstract; nothing to do but extend it
`searching/SearchAlgorithm.java`	given, complete, abstract; nothing to do but extend it
`data/books10.txt`, `books50.txt`, `books150.txt`, `broken.txt`	given sample data

`sorting/` and `searching/` each ship with exactly one file: an abstract contract. **You should add your own concrete class(es) to those folders**, extending those two. See "Your algorithms" below.

To compile and run, from the folder containing `Main.java`:

```
javac Main.java
java Main
```

`javac` finds every other class on its own, by following package names to folders. The scaffold compiles and runs before you've written anything; every `sort`/`find` command will fail with "unknown algorithm" until you've added one.

The REPL

`Main` reads commands from standard input, one per line, until `quit` or end of input. Fields are always one of `id`, `title`, `author`, `year`.

```
add <title>,<author>,<year>      add one book (comma-separated, no commas inside a field)
load <filename>                  read books from a file
sort <field> <algorithm>         sort the catalog by field, using the named algorithm
find <field> <algorithm> <value> search for a book, using the named algorithm
range <field>                    print the smallest and largest value of a field
list                             print every book
count                            print how many books are in the catalog
quit                             exit
```

`<algorithm>` is whatever name you registered for your own class (see below); there's no fixed list. After `sort`/`find`, the REPL prints the operation count and elapsed time directly to the terminal. That's the only place these numbers show up, so read them off the terminal and write down what you need for the design questions.

The contract: `SortingAlgorithm`

Every algorithm extends `SortingAlgorithm` and implements one method: `public void sort(Book[] a)`, sorting `a` in place. Reads and index arithmetic are considered free; what counts is comparisons and array writes. One inherited counter tracks both, and keeping it up to date is your job:

```
protected long operations;   // +1 per comparison, +1 per array write (a swap is +2)
```

Comparisons go through the inherited `order` field, which returns a negative number, zero, or a positive number in the usual way:

```
operations++;
if (order.compare(a[j], a[j + 1]) > 0) {
    // ... swap a[j] and a[j + 1]
    operations += 2;
}
```

`getOperations()` reads it and `resetOperations()` zeroes it. Nothing checks that your counts are right except you: a miscounted algorithm still sorts correctly and still passes `testSort`. If you prefer to wrap the comparing and writing in your own small helper methods so the counting lives in one place, that's fine, just don't count in both places.

Testing. `SortingAlgorithm` gives you `testSort(Book[] a)`, already implemented: it sorts a copy and compares against the JDK's own sort. Check that yours works on:

- the empty array
- a single book
- two books, in both orders
- several books with the same value in the field you're sorting by, but different other fields
- an already-sorted array, and a reverse-sorted one

`BookOrder`: how a field gets chosen

`Book` does **not** implement `Comparable<Book>`. Instead, `order/BookOrder.java` is an interface, extending `java.util.Comparator<Book>`:

```
public interface BookOrder extends Comparator<Book> {
    int compareToQuery(Book b, String query);
    String getName();
}
```

`ById`, `ByTitle`, `ByAuthor`, `ByYear` each implement it, given and complete. Every `SortingAlgorithm`/`SearchAlgorithm` you write takes a `BookOrder` in its constructor and delegates every comparison to it: that's the only place field selection happens.

The contract: `SearchAlgorithm`

Smaller sibling of `SortingAlgorithm`. Searching moves nothing, so there's the same `operations` counter (comparisons only, here), the same inherited `order`, and one method to implement:

```
public abstract int find(Book[] a, String query)   // returns the index of a match, or -1
```

`query` is the raw text typed at the prompt. `order.compareToQuery(book, query)` compares one book's field against it and parses the value for numeric fields, so `find` never has to know what Java type a field is. Count each of those calls, same as in a sort.

Linear search works regardless of sort order. Binary search only gives correct answers if the array is actually ordered by the field you're searching on, and nothing in the given code stops you from running it when that's not true. It will just return a wrong answer, silently, sometimes even a coincidentally correct-looking one.

Your algorithms

No algorithm is given, not even as a template. You write:

- **Linear search and binary search**, one class each.

- **At least two sorting algorithms with different growth rates** (e.g. one $O(n^2)$, one $O(n \log n)$). Which ones is your choice, and you can write more.

Each class goes in its own file in `sorting/` or `searching/`, and starts like this:

```
package sorting;

import catalog.Book;
import order.BookOrder;

public class InsertionSort extends SortingAlgorithm {

    public InsertionSort(BookOrder order) { super(order); }

    @Override
    public void sort(Book[] a) {
        //TODO
    }
}
```

Without the `package` line, `javac` won't find `SortingAlgorithm`; without the imports, it won't find `Book` or `BookOrder`.

`Main` can't know what you'll name your classes, so you register each one yourself in `makeSortAlgorithm`/`makeSearchAlgorithm`, each a `switch` with a `//TODO` marker. Add one `case` per class:

```
case "insertion": return new InsertionSort(order);
```

The string is whatever you type after `sort <field>` or `find <field>` at the prompt.

Catalog: loadFromFile is yours

`Catalog` is given and complete except for one method. It tracks the books (in an `ArrayList<Book>`) and which field they're currently sorted by (`getSortedBy()`, a `String`, `null` until a `sort` succeeds).

```
public void loadFromFile(String path) throws FileNotFoundException   //TODO
```

File format: one book per line, `title,author,year`, no header. Read it with `Scanner`/`File`; both constructors involved throw a checked `FileNotFoundException`, which is why this method declares it. Close what you open. Your choice how to handle a line that doesn't parse (skipping it with a message on `System.err` is reasonable); say what you chose in a comment. `data/broken.txt` has one bad line, for exactly this.

`data/` holds three files of different sizes, `books10.txt`, `books50.txt` and `books150.txt`, with no book repeated between them. Start with the 10-book one, where you can check the result by eye, and move up once it works. Loading all three gives you a 210-book catalog.

`load` appends, it doesn't replace. If you `sort` the catalog and then `load` a small file, the result is a long sorted run with a few new entries tacked on the end: not sorted, but not random either.

So: if the catalog is already sorted and you add a handful of books, which algorithm restores order fastest? Is that still the right choice when the file you loaded is as large as the catalog itself? The data files are sized to let you try both: `load data/books150.txt`, `sort`, then `load data/books10.txt` is the first case; `load data/books50.txt`, `sort`, then `load data/books150.txt` is the second. Compare the operation counts. A hybrid that picks an algorithm based on how much was added is a perfectly acceptable answer, as long as you can say where the threshold is and why.

Range<T>

A small, separate task, not part of the sorting machinery above.

```
public class Range<T extends Comparable<T>> {
    private T min, max;
    public void add(T value) { /* TODO */ }
    public T getMin() { return min; }
```

```
    public T getMax() { return max; }
}
```

After every `add`, `getMin()`/`getMax()` must reflect the smallest/largest value seen so far. Both fields start `null`, so handle the first call.

This backs the `range` command, already wired up in `Main`: it builds a `Range<Integer>` for `id`/`year` and a `Range<String>` for `title`/`author`. Same class, two different type arguments.

Rules

- You may call `Arrays.sort` while testing, e.g. to check your own results, but no submitted algorithm may use it.
- You may add methods, overloads or fields to the given files if they help you. Don't remove anything or change what the existing code does.
- Assume no commas inside a title or author field.

Design questions

Answer these in your submitted file, a paragraph each.

1. Why does `BookOrder` extend `Comparator<Book>` instead of having `Book` implement `Comparable<Book>`?
2. After `load` appends books to an already-sorted catalog, which algorithm do you re-sort with, depending on how many books were added?
3. Why does `Catalog` track which field it's sorted by, instead of just re-sorting before every `find`?
4. What happens when binary search runs on a field the catalog isn't sorted by?
5. Why does `Range<T>` need the bound `T extends Comparable<T>`?
6. Which of your algorithms would you *not* run on a catalog of half a million books, based on the operation counts you observed?

Grading

Code correctness and the required minimum carry most of the weight; the design-question answers are read for whether they show you understand what your code does, not for length.